NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 5 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1: Which high-level API is used in TensorFlow for rapid model prototyping?

Options:

A) PyTorch API

B) Keras API

C) NumPy API

D) TFX API

Answer: B) Keras API

Explanation: Keras serves as the primary high-level API within TensorFlow, providing user-friendly abstractions for constructing and training neural networks. It simplifies prototyping by streamlining layer creation, model compilation, and training execution into clean, high-level commands.

Question 2 : What decorator is used in modern TensorFlow versions to convert eager code to graph execution?

Options:

A) `@tf.graph`

B) `@tf.compile`

C) `@tf.function`

D) `@tf.eager`

Answer: C) `@tf.function`

Indian Institute of Technology Guwahati

Explanation: The `@tf.function` decorator compiles Python functions into optimized TensorFlow computation graphs using AutoGraph. This transformation provides fast graph-mode execution and hardware optimizations while allowing developers to write code imperatively.

Question 3 : How does TensorFlow's gradient tracking behavior during a standard gradient tape call differ from PyTorch's default zero_grad approach?

Options:

A) TensorFlow requires an explicit call to reset gradients.

B) TensorFlow automatically retains all calculated tape states.

C) TensorFlow clears recorded states by default after a gradient call.

D) TensorFlow lacks support for multi-loss tracking entirely.

Answer: C) TensorFlow clears recorded states by default after a gradient call.

Explanation: A standard `tf.GradientTape` automatically releases its recorded operations and memory as soon as `tape.gradient()` is called. In contrast, PyTorch accumulates gradients across operations by default until `zero_grad()` is explicitly called to reset them.

Question 4 : What parameter must be explicitly configured in a TensorFlow gradient tape to successfully evaluate multiple loss functions sequentially without throwing an error?

Options:

A) `accumulate=True`

B) `persistent=True`

C) `retain_graph=True`

D) `trace_gradients=True`

Answer: B) `persistent=True`

Explanation: Setting `persistent=True` keeps the gradient tape from releasing its recorded operations after the first gradient calculation. This enables multiple calls to

Indian Institute of Technology Guwahati

`tape.gradient()` for different loss functions, requiring manual resource cleanup using `del tape`.

Question 5: Which tensor type must be used in TensorFlow to prevent exceptions when trying to perform states or weight updates during training?

Options:

A) `tf.constant`

B) `tf.matrix`

C) `tf.variable`

D) `tf.zeros`

Answer: C) `tf.variable`

Explanation: `tf.Variable` objects represent mutable tensor states whose values can be modified in-place using update ops like `assign_sub`. Standard constants (`tf.constant`) are immutable, causing errors if modified during model weight updates.

Question 6: What is the primary advantage of executing a compiled C++ graph using tf.function over standard eager mode execution in TensorFlow 2.x?

Options:

A) Easier model debugging and line-by-line inspection.

B) Significant execution speedup for computations.

C) Direct execution of native NumPy array slices.

D) Automatic handling of non-persistent gradient tapes.

Answer: B) Significant execution speedup for computations.

Explanation: Graph compilation eliminates Python runtime overhead and enables low-level optimizations like operator fusion and constant folding. This results in significantly faster execution speeds and higher hardware utilization compared to eager evaluation.

Question 7 : How does setting "num_parallel_calls=tf.data.AUTOTUNE" optimize the data pre-processing stage in a TensorFlow input pipeline?

Indian Institute of Technology Guwahati

Options:

A) It dynamically adjusts the batch size dynamically based on memory.

B) It sequentializes data processing to save GPU bandwidth.

C) It triggers a warm-up sequence to minimize initialization time.

D) It automatically balances workloads based on CPU cores and load.

Answer: D) It automatically balances workloads based on CPU cores and load.

Explanation: Using `tf.data.AUTOTUNE` allows TensorFlow to dynamically adjust the level of parallel CPU thread execution at runtime. This removes input bottlenecks and maximizes data throughput automatically without manual thread tuning.

Question 8 : Why does a slow input data pipeline directly reduce overall GPU resource utilization during deep learning training?

Options:

A) It causes the GPU to sit idle waiting for data.

B) It throttles the C++ back-end graph template.

C) It fills the target RAM with synthetic data.

D) It reduces the multi-loss persistent gradient.

Answer: A) It causes the GPU to sit idle waiting for data.

Explanation: GPUs require a continuous pipeline of pre-processed batches to maintain high computational throughput during training steps. If CPU preprocessing is too slow, the GPU stalls waiting for incoming data, leading to underutilized compute resources.

Question 9 : During a multi-epoch training process, what behavior occurs in the second epoch if a dataset applies .map() for pre-processing without using .cache()?

Options:

A) TensorFlow re-executes the map routine calls entirely.

B) The pipeline relies on cached data from the first epoch.

C) The data pipeline automatically throws a syntax warning.

Indian Institute of Technology Guwahati

D) The preprocessing pipeline switches to a C++ graph execution.

Answer: A) TensorFlow re-executes the map routine calls entirely.

Explanation: Without `.cache()`, mapping transformations are evaluated lazily during every single epoch traversal. Using `.cache()` saves the preprocessed dataset in memory or local storage after the first epoch to prevent redundant CPU computation.

Question 10: How can you explicitly prevent unnecessary graph retracing when passing inputs with varying shapes to a @tf.function decorated function?

Options:

A) By defining a custom input_signature with None dimensions.

B) By using regular Python lists instead of TensorFlow tensors.

C) By wrapping the tensor inside a Python while loop construct.

D) By manually clearing the cache before every individual call.

Answer: A) By defining a custom input_signature with None dimensions.

Explanation: Defining dynamic dimensions as `None` within `tf.TensorSpec` allows a single compiled graph to handle variable input dimensions. This prevents TensorFlow from triggering costly graph re-compilations whenever input shapes change across calls.

Question 11 : What is the specific TensorFlow graph equivalent for a standard Python if-else branch structure when building optimized control flows?

Options:

A) `tf.while_loop`

B) `tf.print`

C) `tf.reduce_max`

D) `tf.cond`

Answer: D) `tf.cond`

Indian Institute of Technology Guwahati

Explanation: `tf.cond` builds conditional branch logic directly into the TensorFlow computational graph structure. This ensures both conditional paths can be compiled and correctly evaluated on target hardware like GPUs or TPUs.

Question 12 : During XLA's operator fusion, how are intermediate results stored to prevent High Bandwidth Memory (HBM) traffic?

Options:

A) In cached graphs

B) In global memory

C) In local registers

D) In transient tensors

Answer: C) In local registers

Explanation: XLA fuses multiple consecutive tensor operations into single hardware kernels, keeping temporary results inside ultra-fast on-chip registers. This minimizes memory bandwidth bottlenecks by avoiding unnecessary read/write operations to main HBM memory.

Question 13 : When a model is serialized using TensorFlow's native model.export method for production, what is the primary structural role of the saved "serving signature"

Options:

A) It contains optimizer configurations for resuming continuous training.

B) It encapsulates the metadata required for zero-downtime framework rollback.

C) It explicitly defines the input and output tensors required by inference APIs.

D) It maps network operations directly onto resource-constrained NPU hardware.

Answer: C) It explicitly defines the input and output tensors required by inference APIs.

Explanation: A serving signature defines the exact tensor names, shapes, and data types expected by external inference clients. This creates a clean interface contract that decouples serving environments like TensorFlow Serving from internal graph details.

Indian Institute of Technology Guwahati

Question 14 : If you want to stay strictly within the Google ecosystem and deploy a machine learning model targeting highly resource-constrained mobile apps or edge devices, which framework should you convert your SavedModel to?

Options:

A) ONNX Runtime GPU

B) TensorFlow Light

C) PyTorch TorchScript

D) TensorRT Engine

Answer: B) TensorFlow Light

Explanation: TensorFlow Lite (TFLite) is Google's dedicated framework for executing efficient inference on mobile and micro-edge devices. It applies optimizations like quantization and memory reduction to deliver low latency on embedded hardware.

Question 15 : Which specific file in a standard TensorFlow SavedModel directory is explicitly responsible for maintaining cryptographic hashes to ensure integrity verification during cross-platform model deployment?

Options:

A) `fingerprint.pb`

B) `saved_model.pb`

C) `variables.index`

D) `graph.proto`

Answer: A) `fingerprint.pb`

Explanation: The `fingerprint.pb` file stores cryptographic hashes and metadata describing the exact state of the saved model artifacts. Serving systems use it to verify model integrity and ensure consistency across cross-platform deployments.

Indian Institute of Technology Guwahati